

ShaderAssetChecker使用说明

(1) Tools->AssetChecker->ShaderAssetChecker

使用简介：overview.html是概览页面，也是入口页面。

"Shader复杂度检测报告"，里面可以查看当前项目（Editor模式）下的shader被编译后，计算出顶点片元着色器在通过malioC编译后的复杂度、shader的寄存器数量、是否存在栈内存等信息。当前是按照Mali G51的GPU，FPS：30帧计算35来比较，如果超过的可能存在优化空间。

"损耗性能的shader函数检测报告"，里面可以查看shader编译前代码，包括一层hlsl的包含文件的源码里面，查找到可能存在的敏感耗时函数，因为shader代码里面情况很复杂，所以需要慢慢迭代函数、宏定义等。shader里面的纹理属性、CUBE属性、纹理数组属性等。还有引用shader的材质列表信息、编译后的最少耗时变体函数、最多耗时函数变体采样等信息来综合评估可能存在的优化点。

注意：使用这个功能需要将平台切换到"Android"下，因为编译shader的函数是根据当前平台来编译的。

Complexity↓ //Shader生成的所有指令的累积执行周期数

WorkRegNum↓ //该shader工作使用的寄存器数量，减少提升性能

UniformRegNum↓ //存储着色器可能需要的常量，所有线程都有共享uniform register

StackSpilling↓ //是否有变量被放置到栈内存中，有的话GPU读取是性能消耗较大

16Arithmetic↑ //以16位或更低精度执行的算术运算的百分比。数值越高代表shader性能越好

(2) Shader复杂度的计算公式：(以华为畅玩10Plus Mali-G51为例) 基于Mali架构的GPU，arm官方给出了mali_offline_compiler工具，可以静态扫描shader代码段，来计算出相应的复杂度预算值。因此如果用其他的芯片组，比如：高通骁龙是不行的。

Shader复杂度

- Shader复杂度预算
 - GPU Cycles = Shader Cycles * Pixels
 - 根据目标帧率计算平均每帧每个Pixel的Shader复杂度上限

$$Cycles / Pixel = \frac{ShaderCores \times Frequency}{TargetFPS \times Pixels}$$

- 华为畅玩10Plus (2340 * 1080)
- Mali-G51 MP4 650MHZ

$$Cycles / Pixel = \frac{ShaderCores * Frequency}{TargetFPS * Pixels}$$

$$= \frac{4 * 650M}{60 * 2340 * 1080}$$

$$= 17.1 Cycles per pixel$$

如果通过mali_offline_compiler计算出的目标FPS 60帧的值超过了17.1，复杂度较高，可能存在优化空间。

(3) ShaderAssetChecker—Shader复杂度检测的实现原理：

如果想对项目里面的Shader做复杂度检测，最主要的需要将shader进行编译，按照Unity的shader接口，目前没有直接可用的api去通过脚本编译shader，需要用下面反射接口：

```
180 protected static void OpenCompiledShader(Shader s, int mode, int customPlatformsMask, bool includeAllVariants, bool preprocessOnly, bool stripLineDirectives)
181 {
182     ShaderUtilType.InvokeMember("OpenCompiledShader",
183     BindingFlags.Static | BindingFlags.NonPublic | BindingFlags.InvokeMethod, null, null,
184     new object[] { s, mode, customPlatformsMask, includeAllVariants, preprocessOnly, stripLineDirectives });
185 }
```

这个接口可以顺利的根据当前编辑器所在的环境（DX、OpenGL、iOS）编译shader，所以在执行检测之前，记得先将平台切到Android平台下面，否则编译出来的shader源码分析不了，并且会打开相应的IDE窗口显示（目前没找到不打开窗口的办法，如果有求告知）。根据我们需要我们会选择OpenGL3.2的，设置当前编辑器为OpenGL3.2的版本是下图所示：



通过Unity HUB的附加命令行参数，然后重启编辑器，即可切换到相应平台。

参考连接：<https://docs.unity3d.com/Manual/EditorCommandLineArguments.html>

编译完shader后，这个编译后的shader文件存在于 "ProjectXXX->Temp" 目录下面，找到编译后的shader文件，

Compiled-ZGame-Scene-UnderGround Water.shader	2023/6/19 13:28	SHADER 文件	18 KB
Compiled-ZGame-Scene-StencilMask.shader	2023/6/19 13:29	SHADER 文件	3 KB
Compiled-ZGame-Scene-StencilFill.shader	2023/6/19 13:30	SHADER 文件	7 KB
Compiled-ZGame-Scene-Reflection.shader	2023/6/19 13:42	SHADER 文件	76 KB
Compiled-ZGame-Scene-MaterialBaked.shader	2023/6/19 13:28	SHADER 文件	13 KB
Compiled-ZGame-Scene-Lit.shader	2023/6/19 13:42	SHADER 文件	73,927 KB
Compiled-ZGame-Scene-Leaf.shader	2023/6/19 13:31	SHADER 文件	14,497 KB
Compiled-ZGame-Scene-LavaZone.shader	2023/6/19 13:29	SHADER 文件	10 KB
Compiled-ZGame-Scene-Lambert.shader	2023/6/19 13:52	SHADER 文件	8 KB
Compiled-ZGame-Scene-Grass.shader	2023/6/19 13:25	SHADER 文件	20,389 KB
Compiled-ZGame-Scene-Gem.shader	2023/6/19 13:37	SHADER 文件	203 KB
Compiled-ZGame-Scene-GashaponWater.shader	2023/6/19 13:38	SHADER 文件	15 KB
Compiled-ZGame-Scene-Crystal.shader	2023/6/19 13:24	SHADER 文件	183 KB
Compiled-ZGame-Scene-Cloth.shader	2023/6/19 13:52	SHADER 文件	7,608 KB
Compiled-ZGame-Scene-Boss_EdgeInteraction.shader	2023/6/19 13:34	SHADER 文件	9 KB

可以用文本编辑器打开，里面的内容很多。开头是以下面为文件头：

```
shader_00000000
// Compiled shader for Android

//
// NOTE: This is *not* a valid shader file, the contents are provided just
// for information and for debugging purposes only.
//
// Skipping shader variants that would not be included into build of current scene.
```

```

1 个引用
private void GetShaderComplexityReport(string[] tempAssetPaths, string gpuType, ref List<ShaderComplexityInfo> dic)
{
    int defaultMask = (1 << (int)ShaderCompilerPlatform.GLES3x);
    var tempPath = AssetPath.Substring(0, AssetPath.LastIndexOf('/')) + "/Temp/";

    foreach (var shaderPath in tempAssetPaths)
    {
        var shader = AssetDatabase.LoadAssetAtPath<Shader>(shaderPath);

        if (shaderPath.EndsWith(".shader") && IsCanParseShader(shader.name) && IsNeedAnalysePath(shaderPath))
        {
            OpenCompiledShader(shader, 1, defaultMask, false, false, false);

            var filePath = tempPath + "Compiled-" + shader.name.Replace('/', '-');
            ParseShaderCompiledCode(filePath, shaderPath, gpuType, dic);
        }
    }
}

```

大部分都是以每个shader的不同变体->#ifdef VERTEX ->#ifdef FRAGMENT为单位来罗列的，那我们只需要将其分隔开，拿到相应main() { ... } 的内容片段，并且放到一个文件里，再通过mali_offline_compiler通过命令行，malioc -c xxx.vert malioc -c xxx.frag计算出相应的值，再根据拿到的控制台输出信息，再次分析相关参数，即可拿到我们需要的相关参数。

```

(float, string, string, string, string) CalcShaderComplexity(string destFile, string gpuType, string dataPath)
{
    float cycles = 0; //Shader生成的所有指令的累积执行周期数
    string workRegInfo = ""; //该shader工作使用的寄存器数量，减少提升性能
    string uniformRegCount = ""; //存储着色器可能需要的常量，所有线程都有共享uniform register
    string stackSpilling = ""; //是否有变量被放置到栈内存中，有的话CPU读取是性能消耗较大
    string _16Arithmetic = ""; //以16位或更低精度执行的算术运算的百分比。数值越高代表shader性能越好
    if (File.Exists(destFile))
    {
        var proc = dataPath + "../Tools/mali_offline_compiler/malioc";
        var args = string.Format("{0} -c {1}", "\"" + destFile + "\"", gpuType);
        var info = GetCommandLine(proc, args);
        using (var process = Process.Start(info))
        {
            process.WaitForExit();
            string msg = process.StandardOutput.ReadToEnd();
            if (!string.IsNullOrEmpty(msg))
            {
                if (msg.Contains("Compilation failed."))
                {
                    UnityEngine.Debug.LogError(destFile);
                    UnityEngine.Debug.LogError(msg);
                    UnityEngine.Debug.LogError("-----");
                }
                else
                {
                    var lines = msg.Split('\n');
                    foreach (var item in lines)
                    {
                        if (item.StartsWith("Total instruction cycles:"))
                        {
                            var strs = item.Split(' ');
                            foreach (var str in strs)
                            {
                                if (isNumeric(str, out float v))
                                {
                                    cycles += v;
                                }
                            }
                        }
                        if (item.StartsWith("Work registers"))
                        {
                            var strs = item.Split(':');
                            workRegInfo = strs[1];
                        }
                    }
                }
            }
        }
    }
}

```

```

Configuration
=====
Hardware: Mali-G72 r0p3
Architecture: Bifrost
Driver: r25p0-00rel0
Shader type: OpenGL ES Fragment

Main shader
=====

Work registers: 64
Uniform registers: 26
Stack spilling: false
16-bit arithmetic: 2%

Total instruction cycles:  17.33  0.00  3.75  2.00  A
Shortest path cycles:    11.27  0.00  0.50  0.00  A
Longest path cycles:     17.33  0.00  3.75  2.00  A

A = Arithmetic, LS = Load/Store, V = Varying, T = Texture

Shader properties
=====

Has uniform computation: true
Has side-effects: false
Modifies coverage: true
Uses late ZS test: false
Uses late ZS update: true
Reads color buffer: false

Note: This tool shows only the shader-visible property state.
API configuration may also impact the value of some properties.

```

参考链接: <https://thegamedev.guru/unity-gpu-performance/shader-cost-analysis-mali-offline-compiler/>

(4) ShaderAssetChecker—Shader损耗性能函数检测的实现原理:

这一部分相对来说很简单, 主要是分析在shader编译前的shader源码, 找到预定设置好的耗损函数, 比如pow、tex2D、SAMPLE_TEXTURE2D等, 如果发现有类似的函数存在, 截取这一行, 显示在报告中, 后面根据需求添加了, 纹理属性检测, shader材质引用列表, shader编译后耗时函数分析等功能, 这样如果在shader复杂度那边发现shader可能存在优化的空间后, 在耗损函数这里, 去参考shader编译前后是否存在过多的纹理采样、高复杂度的函数等。

因为部分shader源码里面#include xxxx.hlsl文件, 函数主体在hlsl里面, 因此增加了对内嵌一层hlsl文件的分析, 里面有个通过"Packages/xxx"可以找到实际物理路径的函数:

```

UnityEditor.PackageManager.PackageInfo.FindForAssetPath(path);
pathInfo.resolvedPath

```